

```
!pip uninstall -y bitsandbytes torch torchvision torchaudio
!pip install -q torch==2.5.1+cu121 torchvision==0.20.1+cu121 torchaudio=
!pip install -q transformers datasets accelerate peft trl bitsandbytes==
```

```
from google.colab import drive
drive.mount("/content/drive")
```

Drive already mounted at /content/drive; to attempt to forcibly remount,

```
from pathlib import Path
import os
import sys
import json
import time
import shutil
import gc

import torch
import pandas as pd
```

```
from google.colab import userdata
```

```
os.environ["HF_TOKEN"] = userdata.get("HF_TOKEN")
print("HF_TOKEN cargado:", bool(os.environ.get("HF_TOKEN")))
```

HF_TOKEN cargado: True

```
DRIVE_ROOT = Path("/content/drive/MyDrive/TT2_colab")
```

```
DATA_DIR = DRIVE_ROOT / "data"
SFT_DIR = DATA_DIR / "sft_ready"
OUTPUT_DIR = DRIVE_ROOT / "outputs" / "lora_runs"
OUTPUT_DIR.mkdir(parents=True, exist_ok=True)
```

```
TRAIN_JSONL = SFT_DIR / "feina_repr30_train_sft.jsonl"
VAL_JSONL = SFT_DIR / "feina_repr30_val_sft.jsonl"
TEST_JSONL = SFT_DIR / "feina_repr30_test_sft.jsonl"
```

```
print("TRAIN_JSONL:", TRAIN_JSONL)
print("VAL_JSONL :", VAL_JSONL)
print("TEST_JSONL :", TEST_JSONL)
print("OUTPUT_DIR :", OUTPUT_DIR)
```

```
TRAIN_JSONL: /content/drive/MyDrive/TT2_colab/data/sft_ready/feina_repr30
VAL_JSONL  : /content/drive/MyDrive/TT2_colab/data/sft_ready/feina_repr30
TEST_JSONL : /content/drive/MyDrive/TT2_colab/data/sft_ready/feina_repr30
OUTPUT_DIR : /content/drive/MyDrive/TT2_colab/outputs/lora_runs
```

```

import json

for name, path in {
    "TRAIN_JSONL": TRAIN_JSONL,
    "VAL_JSONL": VAL_JSONL,
    "TEST_JSONL": TEST_JSONL,
}.items():
    print(f"\n=== {name} ===")
    print("Existe:", path.exists())

    if path.exists():
        with open(path, "r", encoding="utf-8") as f:
            first_line = f.readline().strip()

            print("Primera linea vacia?:", first_line == "")

            if first_line:
                sample = json.loads(first_line)
                print("Keys:", list(sample.keys()))
                print("row_id:", sample.get("row_id"))
                print("instruction preview:", str(sample.get("instruction",
                print("output preview:", str(sample.get("output", ""))[:180])

```

```

=== TRAIN_JSONL ===
Existe: True
Primera linea vacia?: False
Keys: ['row_id', 'instruction', 'output', 'text']
row_id: 5
instruction preview: Reescribe en español el siguiente texto con lenguaje
Conserva el significado original y no inventes información.

```

```

Devuelve solo la versión final simplificada.
output preview: El equipo realizó la redacción del libro luego de obtener

```

```

=== VAL_JSONL ===
Existe: True
Primera linea vacia?: False
Keys: ['row_id', 'instruction', 'output', 'text']
row_id: 58
instruction preview: Reescribe en español el siguiente texto con lenguaje
Conserva el significado original y no inventes información.

```

```

Devuelve solo la versión final simplificada.
output preview: Tras este proceso, también se puede obtener un saldo nega

```

```

=== TEST_JSONL ===
Existe: True
Primera linea vacia?: False
Keys: ['row_id', 'instruction', 'output', 'text']
row_id: 92
instruction preview: Reescribe en español el siguiente texto con lenguaje

```

Conserva el significado original y no inventes información.

Devuelve solo la versión final simplificada.

output preview: Muchas personas no tienen el hábito de presupuestar sus g

```
import transformers
import datasets
import peft
import trl
import bitsandbytes as bnb
import accelerate

print("transformers:", transformers.__version__)
print("datasets      :", datasets.__version__)
print("peft          :", peft.__version__)
print("trl           :", trl.__version__)
print("accelerate     :", accelerate.__version__)
print("bnb           :", bnb.__version__)
print("torch          :", torch.__version__)
print("cuda disponible:", torch.cuda.is_available())

if torch.cuda.is_available():
    print("gpu:", torch.cuda.get_device_name(0))
```

```
transformers: 5.7.0
datasets      : 4.8.5
peft          : 0.19.1
trl           : 1.3.0
accelerate     : 1.13.0
bnb           : 0.49.2
torch          : 2.5.1+cu121
cuda disponible: True
gpu: Tesla T4
```

```
MODEL_RUNS = [
    {
        "model_key": "llama3",
        "model_id": "meta-llama/Meta-Llama-3-8B-Instruct",
        "run_name": "lora_llama3_feina_repr30_v1",
    },
    {
        "model_key": "mistral",
        "model_id": "mistralai/Mistral-7B-Instruct-v0.2",
        "run_name": "lora_mistral_feina_repr30_v1",
    },
]

display(pd.DataFrame(MODEL_RUNS))
```

	model_key	model_id	run_name	
0	llama3	meta-llama/Meta-Llama-3-8B-Instruct	lora_llama3_feina_repr30_v1	
1	mistral	mistralai/Mistral-7B-Instruct-v0.2	lora_mistral_feina_repr30_v1	

```

LORA_R = 16
LORA_ALPHA = 32
LORA_DROP_OUT = 0.05

LEARNING_RATE = 2e-4
NUM_TRAIN_EPOCHS = 2
MAX_SEQ_LENGTH = 1024

TARGET_MODULES = ["q_proj", "k_proj", "v_proj", "o_proj"]

PER_DEVICE_TRAIN_BATCH_SIZE = 1
PER_DEVICE_EVAL_BATCH_SIZE = 1
GRADIENT_ACCUMULATION_STEPS = 8
WARMUP_STEPS = 50

```

```

from datasets import load_dataset

data_files = {
    "train": str(TRAIN_JSONL),
    "validation": str(VAL_JSONL),
    "test": str(TEST_JSONL),
}

dataset = load_dataset("json", data_files=data_files)

print(dataset)
print(dataset["train"][0].keys())
print(dataset["train"][0]["text"][:500])

assert dataset["train"].num_rows == 1110, "Train no coincide con repr30"
assert dataset["validation"].num_rows == 238, "Validation no coincide co
assert dataset["test"].num_rows == 238, "Test no coincide con repr30"

print("OK dataset LoRA cargado desde feina_repr30")

```

```

DatasetDict({
  train: Dataset({
    features: ['row_id', 'instruction', 'output', 'text'],
    num_rows: 1110
  })
})

```

```

    })
    validation: Dataset({
        features: ['row_id', 'instruction', 'output', 'text'],
        num_rows: 238
    })
    test: Dataset({
        features: ['row_id', 'instruction', 'output', 'text'],
        num_rows: 238
    })
})
dict_keys(['row_id', 'instruction', 'output', 'text'])
Reescribe en español el siguiente texto con lenguaje más claro y sencillo
Conserva el significado original y no inventes información.

Devuelve solo la versión final simplificada.

Texto:
Luego de haber obtenido la anterior información y haber hecho acopio de l

Versión simplificada:El equipo realizó la redacción del libro luego de ob
OK dataset LoRA cargado desde feina_repr30

```

```

from transformers import BitsAndBytesConfig

compute_dtype = torch.float16

bnb_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_quant_type="nf4",
    bnb_4bit_use_double_quant=True,
    bnb_4bit_compute_dtype=torch.float16,
)

print("compute_dtype:", compute_dtype)
bnb_config

```

```

compute_dtype: torch.float16
BitsAndBytesConfig {
  "_load_in_4bit": true,
  "_load_in_8bit": false,
  "bnb_4bit_compute_dtype": "float16",
  "bnb_4bit_quant_storage": "uint8",
  "bnb_4bit_quant_type": "nf4",
  "bnb_4bit_use_double_quant": true,
  "llm_int8_enable_fp32_cpu_offload": false,
  "llm_int8_has_fp16_weight": false,
  "llm_int8_skip_modules": null,
  "llm_int8_threshold": 6.0,
  "load_in_4bit": true,
  "load_in_8bit": false,
  "quant_method": "bitsandbytes"
}

```

```
from transformers import (
    AutoTokenizer,
    AutoModelForCausalLM,
    TrainingArguments,
)
from peft import prepare_model_for_kbit_training, LoraConfig, TaskType
from trl import SFTTrainer

def run_lora_training(run_cfg: dict, dataset):
    model_key = run_cfg["model_key"]
    model_id = run_cfg["model_id"]
    run_name = run_cfg["run_name"]

    run_dir = OUTPUT_DIR / run_name
    run_dir.mkdir(parents=True, exist_ok=True)

    final_dir = run_dir / "final_adapter"
    final_dir.mkdir(parents=True, exist_ok=True)

    print("\n" + "=" * 90)
    print("Iniciando corrida")
    print("model_key:", model_key)
    print("model_id :", model_id)
    print("run_name :", run_name)
    print("=" * 90)

    tokenizer = AutoTokenizer.from_pretrained(
        model_id,
        use_fast=True,
        token=os.environ.get("HF_TOKEN")
    )

    if tokenizer.pad_token is None:
        tokenizer.pad_token = tokenizer.eos_token
    tokenizer.padding_side = "right"

    print("pad_token:", tokenizer.pad_token)
    print("eos_token:", tokenizer.eos_token)

    t0_load = time.perf_counter()
    model = AutoModelForCausalLM.from_pretrained(
        model_id,
        quantization_config=bnb_config,
        dtype=torch.float16,
        device_map="auto",
        token=os.environ.get("HF_TOKEN")
    )
    model.config.use_cache = False
    t1_load = time.perf_counter()

    print(f"Tiempo cargando modelo: {t1_load - t0_load:.2f} s")
```

```
model = prepare_model_for_kbit_training(model)
model.gradient_checkpointing_enable()
print("Modelo preparado para k-bit training")

peft_config = LoraConfig(
    task_type=TaskType.CAUSAL_LM,
    inference_mode=False,
    r=LORA_R,
    lora_alpha=LORA_ALPHA,
    lora_dropout=LORA_DROP_OUT,
    bias="none",
    target_modules=TARGET_MODULES,
)

print(peft_config)

training_args = TrainingArguments(
    output_dir=str(run_dir),
    num_train_epochs=NUM_TRAIN_EPOCHS,
    per_device_train_batch_size=PER_DEVICE_TRAIN_BATCH_SIZE,
    per_device_eval_batch_size=PER_DEVICE_EVAL_BATCH_SIZE,
    gradient_accumulation_steps=GRADIENT_ACCUMULATION_STEPS,
    learning_rate=LEARNING_RATE,
    weight_decay=0.0,
    warmup_steps=WARMUP_STEPS,
    lr_scheduler_type="cosine",
    logging_steps=10,
    eval_strategy="epoch",
    save_strategy="epoch",
    save_total_limit=2,
    fp16=False,
    bf16=False,
    max_grad_norm=0.3,
    report_to="none",
    remove_unused_columns=False,
    dataloader_pin_memory=False,
)

trainer = SFTTrainer(
    model=model,
    args=training_args,
    train_dataset=dataset["train"],
    eval_dataset=dataset["validation"],
    processing_class=tokenizer,
    peft_config=peft_config,
    formatting_func=lambda ex: ex["text"],
)

print("Trainer creado correctamente")
```

```

t0_train = time.perf_counter()
train_result = trainer.train()
t1_train = time.perf_counter()

train_minutes = (t1_train - t0_train) / 60
print(f"Tiempo total entrenamiento: {train_minutes:.2f} min")
print(train_result)

trainer.model.save_pretrained(final_dir)
tokenizer.save_pretrained(final_dir)
print("Adapter guardado en:", final_dir)

run_metadata = {
    "run_name": run_name,
    "model_key": model_key,
    "model_id": model_id,
    "dataset_prefix": "feina_repr30",
    "train_jsonl": str(TRAIN_JSONL),
    "val_jsonl": str(VAL_JSONL),
    "test_jsonl": str(TEST_JSONL),
    "train_rows": len(dataset["train"]),
    "val_rows": len(dataset["validation"]),
    "test_rows": len(dataset["test"]),
    "max_seq_length": MAX_SEQ_LENGTH,
    "lora_r": LORA_R,
    "lora_alpha": LORA_ALPHA,
    "lora_dropout": LORA_DROPOUT,
    "target_modules": TARGET_MODULES,
    "learning_rate": LEARNING_RATE,
    "epochs": NUM_TRAIN_EPOCHS,
    "per_device_train_batch_size": PER_DEVICE_TRAIN_BATCH_SIZE,
    "per_device_eval_batch_size": PER_DEVICE_EVAL_BATCH_SIZE,
    "gradient_accumulation_steps": GRADIENT_ACCUMULATION_STEPS,
    "load_in_4bit": True,
    "bnb_4bit_quant_type": "nf4",
    "bnb_4bit_use_double_quant": True,
    "compute_dtype": str(compute_dtype),
    "train_runtime_minutes": train_minutes,
    "final_adapter_dir": str(final_dir),
}

meta_path = run_dir / "run_metadata.json"
with open(meta_path, "w", encoding="utf-8") as f:
    json.dump(run_metadata, f, ensure_ascii=False, indent=2)

print("Metadata guardada en:", meta_path)

sample_text = dataset["validation"][0]["instruction"]
inputs = tokenizer(sample_text, return_tensors="pt").to(trainer.model.device)

with torch.no_grad():

```



```
        outputs = trainer.model.generate(
            **inputs,
            max_new_tokens=256,
            do_sample=False,
            pad_token_id=tokenizer.eos_token_id,
        )

    decoded = tokenizer.decode(outputs[0], skip_special_tokens=True)

    sample_out_path = run_dir / "sample_generation.txt"
    with open(sample_out_path, "w", encoding="utf-8") as f:
        f.write(decoded)

    print("\nEjemplo de salida:")
    print(decoded[:1200])
    print("\nSample guardado en:", sample_out_path)

    # liberar memoria
    del trainer
    del model
    gc.collect()
    if torch.cuda.is_available():
        torch.cuda.empty_cache()

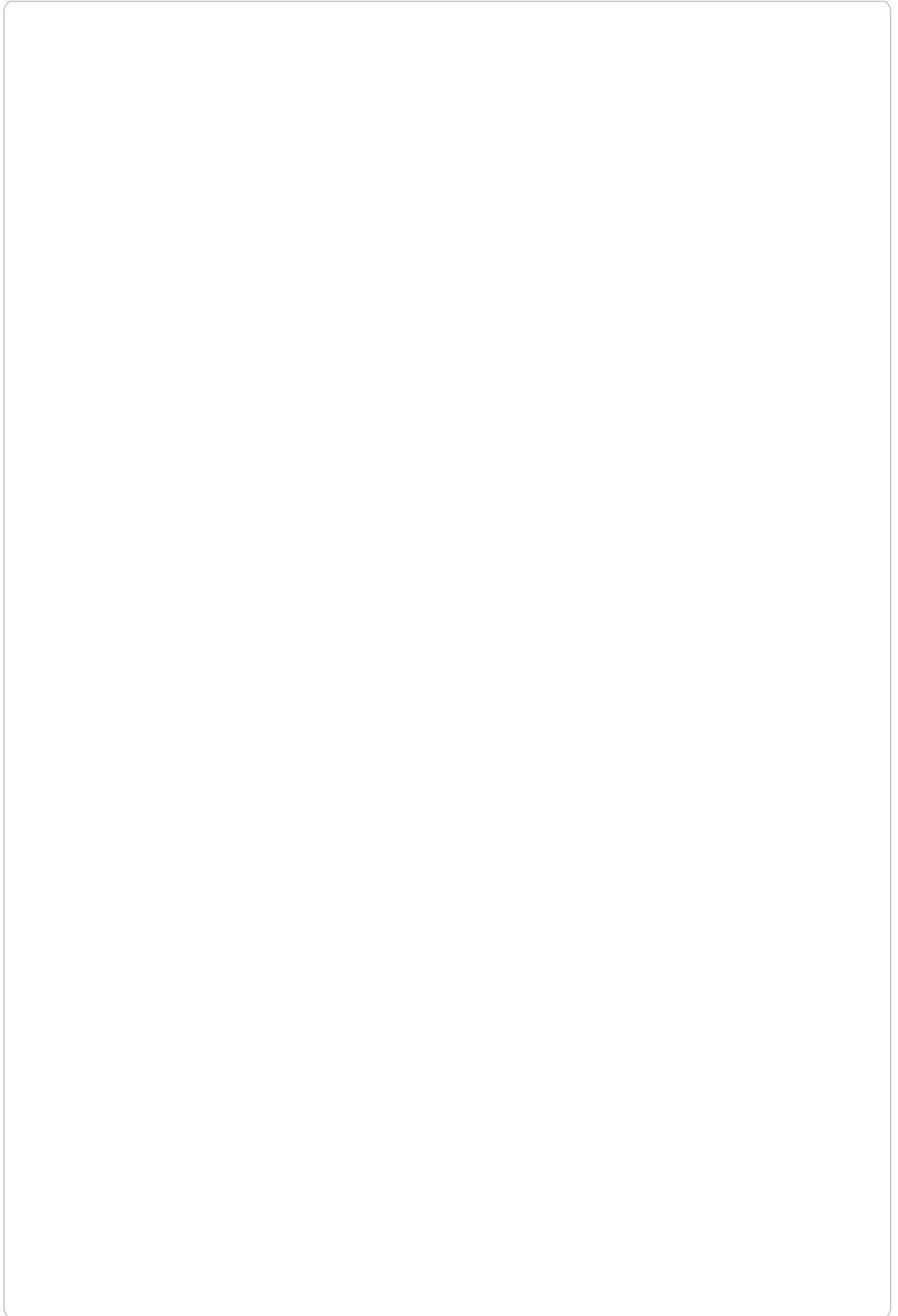
    return run_metadata
```

```
pip install -U bitsandbytes>=0.46.1
```

```
all_run_metadata = []

for run_cfg in MODEL_RUNS:
    run_meta = run_lora_training(run_cfg, dataset)
    all_run_metadata.append(run_meta)

runs_df = pd.DataFrame(all_run_metadata)
display(runs_df)
```



```

=====
ndo corrida
key: llama3
id : meta-llama/Meta-Llama-3-8B-Instruct
me : lora_llama3_feina_repr30_v1
=====
ken: <|eot_id|>
ken: <|eot_id|>
afetensors.index.json:      23.9k/? [00:00<00:00, 482kB/s]

ad complete: 100%      16.1G/16.1G [07:33<00:00, 39.0MB/s]

j 4 files: 100%                                4/4 [07:33<00:00, 453.66s/it]

weights: 100%                                291/291 [01:09<00:00, 6.09it/s]

on_config.json: 100%                                187/187 [00:00<00:00, 21.4kB/s]

cargando modelo: 525.10 s
preparado para k-bit training
nfig(task_type=<TaskType.CAUSAL_LM: 'CAUSAL_LM'>, peft_type=<PeftType
j formatting function to train dataset: 100%  1110/1110 [00:00<00:00, 8239.02 examples/s]

EOS to train dataset: 100%                    1110/1110 [00:00<00:00, 10080.80 examples/s]

ng train dataset: 100%                        1110/1110 [00:01<00:00, 989.62 examples/s]

j formatting function to eval dataset: 100%  238/238 [00:00<00:00, 4760.59 examples/s]

EOS to eval dataset: 100%                    238/238 [00:00<00:00, 4249.71 examples/s]

ng eval dataset: 100%                        238/238 [00:00<00:00, 1023.70 examples/s]

formers] The tokenizer has new PAD/BOS/EOS tokens that differ from
r creado correctamente
[278/278 44:12, Epoch 2/2]

Training Loss  Validation Loss
-----
0.955045          0.894690

0.867281          0.889716

total entrenamiento: 44.36 min
output(global_step=278, training_loss=1.0223954838814495, metrics={'
formers] Both `max_new_tokens` (=256) and `max_length` (=4096) seem
r guardado en: /content/drive/MyDrive/TT2_colab/outputs/lora_runs/l
ta guardada en: /content/drive/MyDrive/TT2_colab/outputs/lora_runs/
formers] Ignoring clean_up_tokenization_spaces=True for BPE tokeniz

o de salida:
e en español el siguiente texto con lenguaje más claro y sencillo
va el significado original y no inventes información.

ve solo la versión final simplificada.

```

uede terminar también con un saldo negativo, lo cual significa que

n simplificada: Aquí puede terminar con un saldo negativo, lo que si
 guardado en: /content/drive/MyDrive/TT2_colab/outputs/lora_runs/lo

```
=====
ndo corrida
key: mistral
id : mistralai/Mistral-7B-Instruct-v0.2
me : lora_mistral_feina_repr30_v1
=====
```

on: 100% 596/596 [00:00<00:00, 49.8kB/s]

r_config.json: 2.10k/? [00:00<00:00, 122kB/s]

r.json: 1.80M/? [00:00<00:00, 14.4MB/s]

r.model: 100% 493k/493k [00:00<00:00, 2.46MB/s]

tokens_map.json: 100% 414/414 [00:00<00:00, 47.9kB/s]

ken: </s>

ken: </s>

afetensors.index.json: 25.1k/? [00:00<00:00, 2.26MB/s]

ad complete: 100% 14.5G/14.5G [06:07<00:00, 77.3MB/s]

g 3 files: 100% 3/3 [06:07<00:00, 152.89s/it]

weights: 100% 291/291 [01:02<00:00, 6.12it/s]

on_config.json: 100% 111/111 [00:00<00:00, 12.3kB/s]

cargando modelo: 431.88 s

preparado para k-bit training

nfig(task_type=<TaskType.CAUSAL_LM: 'CAUSAL_LM'>, peft_type=<PeftTy

EOS to train dataset: 100% 1110/1110 [00:00<00:00, 12799.82 examples/s]

ng train dataset: 100% 1110/1110 [00:00<00:00, 1913.54 examples/s]

EOS to eval dataset: 100% 238/238 [00:00<00:00, 6073.97 examples/s]

ng eval dataset: 100% 238/238 [00:00<00:00, 1709.18 examples/s]

formers] The tokenizer has new PAD/BOS/EOS tokens that differ from
 r creado correctamente

[278/278 45:03, Epoch 2/2]

Training Loss Validation Loss

0.786466	0.736286
0.661337	0.740271

total entrenamiento: 45.21 min

output(global_step=278, training_loss=0.7964872696416841, metrics={'

r guardado en: /content/drive/MyDrive/TT2_colab/outputs/lora_runs/lo

ta guardada en: /content/drive/MyDrive/TT2_colab/outputs/lora_runs/

o de salida:

```

from pathlib import Path

OUTPUT_DIR = Path("/content/drive/MyDrive/TT2_colab/outputs/lora_runs")

for run_name in [
    "lora_llama3_feina_repr30_v1",
    "lora_mistral_feina_repr30_v1",
]:
    run_dir = OUTPUT_DIR / run_name
    print(f"\n=== {run_name} ===")
    print("Existe carpeta:", run_dir.exists())
    if run_dir.exists():
        print("Contenido:", [p.name for p in run_dir.iterdir()])
        print("Tiene final_adapter:", (run_dir / "final_adapter").exists())
        print("Tiene run_metadata.json:", (run_dir / "run_metadata.json").exists())
        print("Tiene sample_generation.txt:", (run_dir / "sample_generation.txt").exists())

```

```

lora_mistral_feina_repr30_v1      mistral      mistralai/Mistral-7B-Instruct-v0.2      feina_repr30      /content/drive/MyDrive/TT2_colab/outputs/lora_runs/
=== lora_llama3_feina_repr30_v1 ===
Existe carpeta: True
Contenido: ['final_adapter', 'checkpoint-139', 'checkpoint-278', 'run_metadata.json', 'sample_generation.txt']
Tiene final_adapter: True
Tiene run_metadata.json: True
Tiene sample_generation.txt: True

=== lora_mistral_feina_repr30_v1 ===
Existe carpeta: True
Contenido: ['final_adapter', 'checkpoint-139', 'checkpoint-278', 'run_metadata.json', 'sample_generation.txt']
Tiene final_adapter: True
Tiene run_metadata.json: True
Tiene sample_generation.txt: True

```

```

from pathlib import Path
import json
import pandas as pd

OUTPUT_DIR = Path("/content/drive/MyDrive/TT2_colab/outputs/lora_runs")

all_run_metadata = []

for run_dir in OUTPUT_DIR.iterdir():
    if run_dir.is_dir():
        meta_path = run_dir / "run_metadata.json"
        if meta_path.exists():
            with open(meta_path, "r", encoding="utf-8") as f:
                meta = json.load(f)
            all_run_metadata.append(meta)

runs_df = pd.DataFrame(all_run_metadata)
display(runs_df)

```

```
runs_summary_path = OUTPUT_DIR / "runs_summary.csv"
runs_df.to_csv(runs_summary_path, index=False, encoding="utf-8")
print("Resumen guardado en:", runs_summary_path)

print("\nContenido de OUTPUT_DIR:")
for p in OUTPUT_DIR.iterdir():
    print("-", p.name)
```